

03 — UI/UX Specification

Product: Endpoint IQ **Document:** UI/UX Specification

1. Design Principles

1. **Developer/DevOps tool, not a chatbot.** Endpoint IQ's UI should read like Postman, Grafana, or a CI dashboard — dense, structured, data-forward. No conversational chat bubbles for AI output; AI insights are presented as structured cards/sections within the results workflow.
2. **Test and failure visibility first.** Per the source document's explicit product-behavior guidance, the UI must prioritize test visibility, failure visibility, and actionable AI recommendations over decoration.
3. **AI is a suggestion, not an authority.** Every AI-generated element is visually labeled (icon + "AI-generated" tag) and never rendered as if it were a verified test result.
4. **Calm, low-animation interface.** Status changes and data loads should be clear without being distracting; avoid excessive motion (explicit requirement).
5. **Consistency over novelty.** Reuse the same table, card, and status-indicator patterns across every workflow stage so users build muscle memory.

2. Layout

Standard app shell: **fixed left sidebar + top header + scrollable content area.**

```
flowchart TB
    subgraph Shell
        H[Header: project switcher, user menu, global status]
        S[Sidebar: workflow steps for current project]
        C[Content area: page content]
    end
    H --- S
    S --- C
```

- **Sidebar:** project-scoped navigation reflecting the workflow: Overview, Endpoints, Roles, Credentials, Test Runs, Results, Reports. Collapsible on smaller viewports.
- **Header:** current project name/switcher, breadcrumb of workflow stage, user/account menu, connection/run status indicator.
- **Content area:** max-width constrained for readability on wide screens; tables and code viewers may use full width.

3. Navigation

- Primary nav = sidebar, scoped to the currently selected project once a project exists.
- A top-level "Projects" view (no project selected) lists all projects and is the landing page after login.
- Breadcrumbs: Projects / {Project Name} / {Workflow Step}.
- Within a test run, a secondary tab bar switches between **Overview / Endpoint Results / Failed APIs / AI Analysis / Report**.

4. Typography

Role	Suggested treatment
Page titles	Semi-bold, larger size, sans-serif
Section headers	Medium weight, uppercase-tracking optional for sidebar section labels
Body text	Regular weight, high-legibility sans-serif
Code / JSON / request-response / paths	Monospace font throughout (critical — this is a technical product)

Open Decision: Specific font family/type scale is not defined in the source document. Recommend a standard system/sans-serif stack (e.g. Inter) for UI text and a monospace stack (e.g. JetBrains Mono / Menlo) for code, but this is a design-system decision outside the architecture doc’s scope.

5. Colors

Color is primarily used for **status communication**, not decoration. A neutral base palette (light and/or dark mode) with a fixed semantic status palette:

Status	Suggested color role
Success / Passed	Green
Failed	Red
Warning	Amber/Yellow
Running	Blue (often with subtle pulse, not spin animation)
Pending / Queued	Gray
Critical (security finding)	Deep red / distinct from generic “Failed”
AI Recommendation	Purple/violet accent — visually distinct from pass/fail colors so AI content is never confused with raw test status

Open Decision: Exact hex values, light/dark theme support are not specified in the source. Recommend defining a design-token file (tokens.ts or CSS variables) so colors are centrally managed — see 02-Frontend-Architecture.md folder structure (styles/tokens can live under utils/ or a dedicated styles/ folder).

6. Spacing

Use a consistent spacing scale (e.g. 4px base unit: 4/8/12/16/24/32/48). Tables and cards use consistent internal padding (e.g. 12–16px) so the dense data views (endpoint lists, result tables) don’t feel cramped.

7. Cards

Used for: project summary, test-run summary stats, AI analysis findings, report summary. Each card has a title, optional status badge, and content region. AI-analysis cards additionally carry a small “AI-generated” tag in the header (see §16).

8. Tables

Primary pattern for endpoint lists and test results. Requirements: - Sortable columns (method, path, status, response time). - Filterable by status (Passed/Failed/Warning), role, and method. -

Row click opens detail (failed endpoint detail, or endpoint spec detail). - Sticky header on scroll for long tables. - Method column uses colored badges (GET/POST/PUT/DELETE) consistent with common API-tool conventions.

9. Buttons

- **Primary** — main workflow action (Import, Start Test Run, Download Report). One primary button per view.
- **Secondary** — supporting actions (Cancel, Edit, Add Role).
- **Destructive** — delete/remove actions, always paired with a confirmation modal.
- **Icon buttons** — compact actions in table rows (view, copy, download).

10. Forms

Used in: Project creation, Role configuration, Credential configuration, Swagger import. - In-line validation messages under each field. - Clear required-field marking. - Credential fields (username/password) always use masked password inputs; a “test connection” affordance is recommended (see Open Decision below) but not defined in source.

Open Decision: Whether the frontend should offer a “Test Credentials” / “Verify Login” action before starting a full run is not defined in the source architecture, though the Auth Service’s login flow (§4.3) would technically support it. Flag as a **Proposed API Contract** in 05-Frontend-API-Integration.md if desired.

11. Inputs

Standard text, password (masked), URL (for Swagger import, with inline format validation), and select/dropdown inputs. All inputs support disabled and error states.

12. Dropdowns

Used for: role selector, project switcher, method/status filters. Standard single-select; multi-select for status filters in the results table.

13. Tabs

Used within a test run detail view to switch between Overview / Results / Failed APIs / AI Analysis / Report, and within a failed-endpoint detail to switch between Request / Response / Headers / Diff.

14. Modals

Used for: confirmations (delete project, cancel run), credential entry (optional modal vs. inline form — implementation choice), and quick endpoint spec preview.

15. Toasts

Used for transient feedback: “Import started,” “Test run started,” “Report ready,” and non-blocking errors (“Failed to save role — try again”). Auto-dismiss with a manual close option; errors persist slightly longer than success toasts.

16. Badges / Status Indicators

A single shared `StatusPill` component drives all status displays across the app, parameterized by the states below:

State	Used for
Success	Passed test / completed run
Failed	Failed test / failed run
Warning	Partial pass, flaky result, deprecated endpoint
Running	In-progress test run
Pending	Queued job / not-yet-run endpoint
Critical	Security finding from AI analysis
AI Recommendation	Any AI-generated content block

17. Code Blocks / JSON Viewer

A shared `JsonViewer/CodeBlock` component (collapsible tree, syntax highlighting, copy-to-clipboard) is used everywhere raw data is shown: OpenAPI schema snippets, request/response bodies, headers. This is a core, high-reuse component — build it once in `components/` (see `02-Frontend-Architecture.md`).

18. Request/Response Viewer

A composite component (built from `Tabs` + `JsonViewer` + `CodeBlock`) showing, for a given test result: Request (method, path, headers, body), Response (status, headers, body), and — for failures — an Expected vs. Actual diff view. Full detail in **07-Frontend-Test-Results-and-AI-Report-UI.md**.

19. AI Analysis Cards

Distinct visual treatment (purple/violet accent, “AI-generated” tag, brain/sparkle icon — icon choice is a design decision) containing: summary, root cause, recommended fix, and a “Developer Fix Prompt” block (copyable). Always includes a short disclaimer treatment (e.g. muted caption) reflecting the source document’s explicit instruction that AI output is a suggestion for human review, not an automated authority.

20. Report Sections

The report view (pre-download) mirrors the downloadable file’s structure: Pass/fail summary per endpoint and role, error details and response diffs, AI recommendations (highlighted at top, per §4.9 of the architecture), and a download button.

21. Visual States

State	Applies to	Treatment
Success	Passed tests, completed runs	Green StatusPill
Failed	Failed tests, failed runs	Red StatusPill
Warning	Partial/flaky results	Amber StatusPill

State	Applies to	Treatment
Running	Active test run	Blue StatusPill, subtle non-spinning progress indicator
Pending	Queued jobs	Gray StatusPill
Critical	Security findings	Distinct deep-red StatusPill, separate from generic Failed
AI Recommendation	Any AI content	Purple accent + tag

22. Loading States

Skeleton screens for: project list, endpoint list, results table. Simple spinners only for short, discrete actions (button submit).

23. Skeleton States

Table skeletons (shimmer rows) for endpoint list, results table, and report list while data loads.

24. Empty States

- No projects yet → prompt to create first project.
- No endpoints yet → prompt to import a Swagger spec.
- No roles/credentials yet → prompt to add one.
- No test runs yet → prompt to start one.
- No AI findings (all passed) → positive empty state, not an error.

25. Error States

- Import failed (bad spec) → clear message + link to fix and retry (per §10 Risks, “Bad OpenAPI Spec... give user clear error message”).
- Test run failed to start → actionable error with retry.
- Network/server error → generic retry state, distinct from validation errors.

26. Confirmation States

Required for: deleting a project, deleting a role/credential, canceling a running test run. Standard ConfirmDialog component with explicit destructive-action styling.

27. Responsive Behavior

Endpoint IQ is primarily a desktop/wide-viewport tool (dense tables, code viewers), but the shell should degrade gracefully to tablet width (collapsible sidebar, horizontally scrollable tables). Full mobile optimization is **not** implied by the source document.

Open Decision: Mobile-first support is not called for in the architecture; recommend “responsive down to tablet, not optimized for phone” as the working target unless product decides otherwise.

28. Accessibility

- All status information conveyed by color must also have a text/icon equivalent (not color-only).
- Full keyboard navigation for forms, tables (row focus), and modals (focus trap).
- Sufficient contrast ratios (WCAG AA) for status pill text/background pairs.
- `aria-live` regions for test-run progress updates so screen readers announce state changes.

Cross-references: 04-Frontend-Pages-and-Routes.md (where each pattern is used), 07-Frontend-Test-Results-and-AI-Report-UI.md (results/AI UI detail).